

CS311 Operating Systems
Assignment 01 - UPDATED - : Mini Shell
Due : 23h55 Sunday 20th September, 2026.

FCS&E
GIKI

Fall 2026

1 Objectives

- (a) Test the student's C programming skills.
- (b) Test the student's ability to write clean readable code.
- (c) Test the student's ability to use Linux System Calls.
- (d) Test the student's ability do error handling in code.
- (e) Test the student's ability to follow written instructions.

2 Description

In this assignment you will be doing C programming, using Linux system calls, using Makefiles to structure your build system, and debugging using `gdb`.

The evaluation based on this assignment will weigh 5% of your course grade.

Note: Please start early so you have time to learn things which you don't.

3 Tasks

Q1. Make a mini shell ¹

We'll make a mini shell of our own. Let's call it `mysh` MY SHell. The shell would:

- (a) When run, display a cool prompt of your choosing and prompt the user for input. Just like you see in Linux/Windows command line.
- (b) When given a command with arguments, it would create a child process and run the command with its arguments.
- (c) By default it would wait for the command to finish before displaying the prompt again. ²

¹read about `fork()`, `exec()`, `wait()`, `waitpid()`, `getpid()`, `getenv()`, `setenv()`, `unsetenv()`, `int main(int argc, char* argv[], char* envp[])`, `kill()`

²you can't use the `system()` syscall here. Have to use `fork()`, `exec()` to run the command as a child process of the shell.

- (d) Most Linux commands are separate programs that the shell runs, however some of them are shell builtins - you can google the difference. Your shell should support the builtins `pwd`, `cd`, The should behave the same as they do in `bash`. Your program should implement whatever internal state it needs for their functionality.
- (e) Internally your shell would maintain information about the processes it has created and that are currently alive. Whenever a user runs the `ps` command, instead of running the normal `ps` program, it should run its own version of it. It shall check its internal data structure and list the names of the processes and their PIDs on the screen. Ditto for the `kill` command, it should just terminate the process whose PID is provided as argument to `kill` and expunge its record from its internal data structures.
- (f) Bash allows you to run multiple commands in one line when you separate them by the ampersand (&) character. If you type only one command followed by &, it runs the command in back ground and displays the prompt immediately. `mysh` should imitate the same behaviour.
- (g) By default the commands run on via your shell shall take input from keyboard and write their output on the screen, however when given a command followed by a redirection operator (`>`, `>>`) followed by a filename, it should redirect the output of the command to the said file by overwriting or appending to it respectively.
- (h) Similarly, when given a command followed by a redirection operator (`<`) followed by a filename, it should make the given command read its input from the given file instead of the Standard Input.
- (i) Linux shells also let you redirect the output of one command to the input of another using the pipe operator, i.e., `cat file.txt | wc -l`. While those shells let you chain arbitrary many commands in this fashion, yours shall let connect two commands at least in this manner. ³

4 Some Instructions

- Your shell shall implement only the functionality mentioned in this document, nothing extra!
- It is important for you to learn to use Open Source repositories. For the reading user input on the command line, use the `linenoise` library. ⁴
- Use Make or CMake as the build system for your project,

5 Error Checking

- When working with Linux System calls, it is extremely important that you always check the return values of the system calls to verify whether the call was successful or not. The function documentation would usually contain all the information you will need. You should always read the function documentation. You might also want to read about the use of `errno`, `perror()`, and `strerror()`.
- Your programs should guard against invalid user input and in case of an invalid input, it should print a helpful error message.

³see the documentation for `pipe()` and `dup2()` system calls

⁴<https://github.com/antirez/linenoise>

- You would lose marks if your program crashes during use. Use `gdb` to find any bug in your program.
- It is the programmer's responsibility to free any system resources i.e. memory, file descriptors, etc., they have acquired from the system. Your program should always free system resources once it is done using them. `Valgrind` is a tool that you can run to check for memory leaks, etc.
- Code should be properly indented, readable and commented.
- You should always compile your code using the `-Wall` option.

6 Submission Instructions

1. Your submission should consist of `.c` files, `.h` files and a make file.
2. Prefix the names of all your submitted files by `CS311_A01_2034XXX`, where `XXX` stands for the last 3 digits for your Giki registration number.

7 Rubric

Category	Marks
Followed submission insns	20 marks
Code was readable + Compiled without warnings + Program handles errors well	20 marks
Functionality working properly	60 marks
Total	100 marks